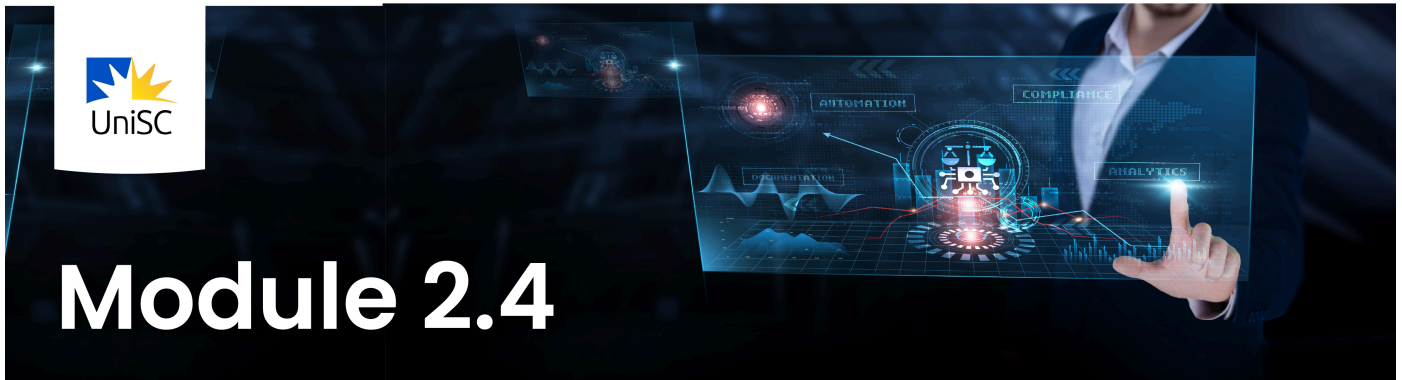


2.4: Estimating and Analysing Volatility Clustering [↕]



In Section 2.3, you learned that volatility is dynamic and that financial markets exhibit **volatility clustering**.

In this section, the central question is this: **Can we see volatility clustering in real financial data? And how do we measure it?**

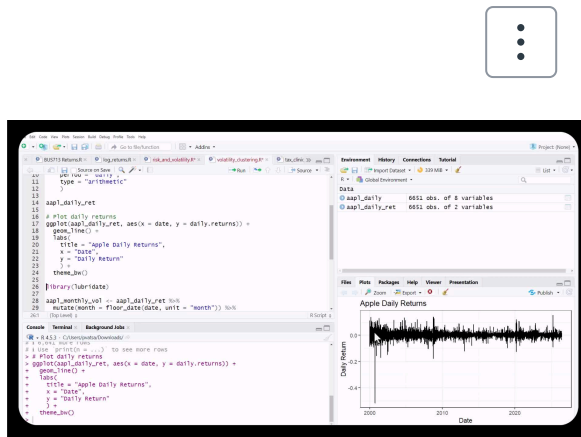
We proceed in four steps:

1. Visually identify clusters in daily returns
2. Measure realised volatility
3. Quantify clustering using squared returns
4. Use ACF plots to assess persistence formally

We use Apple as our main example.

☐ Watch

[Volatility clustering and realized volatility](#)



[R Script \(https://learn.usc.edu.au/courses/35359/files/3294295?wrap=1\)](https://learn.usc.edu.au/courses/35359/files/3294295?wrap=1)
(https://learn.usc.edu.au/courses/35359/files/3294295/download?download_frd=1)



Details

Transcript



🔍 Search in transcript

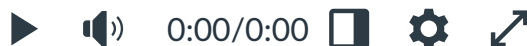


0:00 In this video we will talk about the volatility

0:02 clustering in Apple stock. So once again start by

0:07 loading the packages using the library function.

0:09 And by now you should be quite familiar with the



Details

Transcript



Squared returns and autocorrelation function

Visualising Volatility Clusters in Daily Returns

We begin with daily returns. Clustering should be visible before we calculate anything.

Step 1: Compute daily returns

```
library(tidyverse)

library(tidyquant)

aapl_daily <- tq_get("AAPL", from = "2000-01-01")

aapl_daily_ret <- aapl_daily %>%

  tq_transmute(

    select = adjusted,

    mutate_fun = periodReturn,

    period = "daily",

    type = "arithmetic"

  )
```

The result is a time series of daily returns.

Plot daily returns

```
ggplot(aapl_daily_ret, aes(x = date, y = daily.returns)) +

  geom_line() +

  labs(

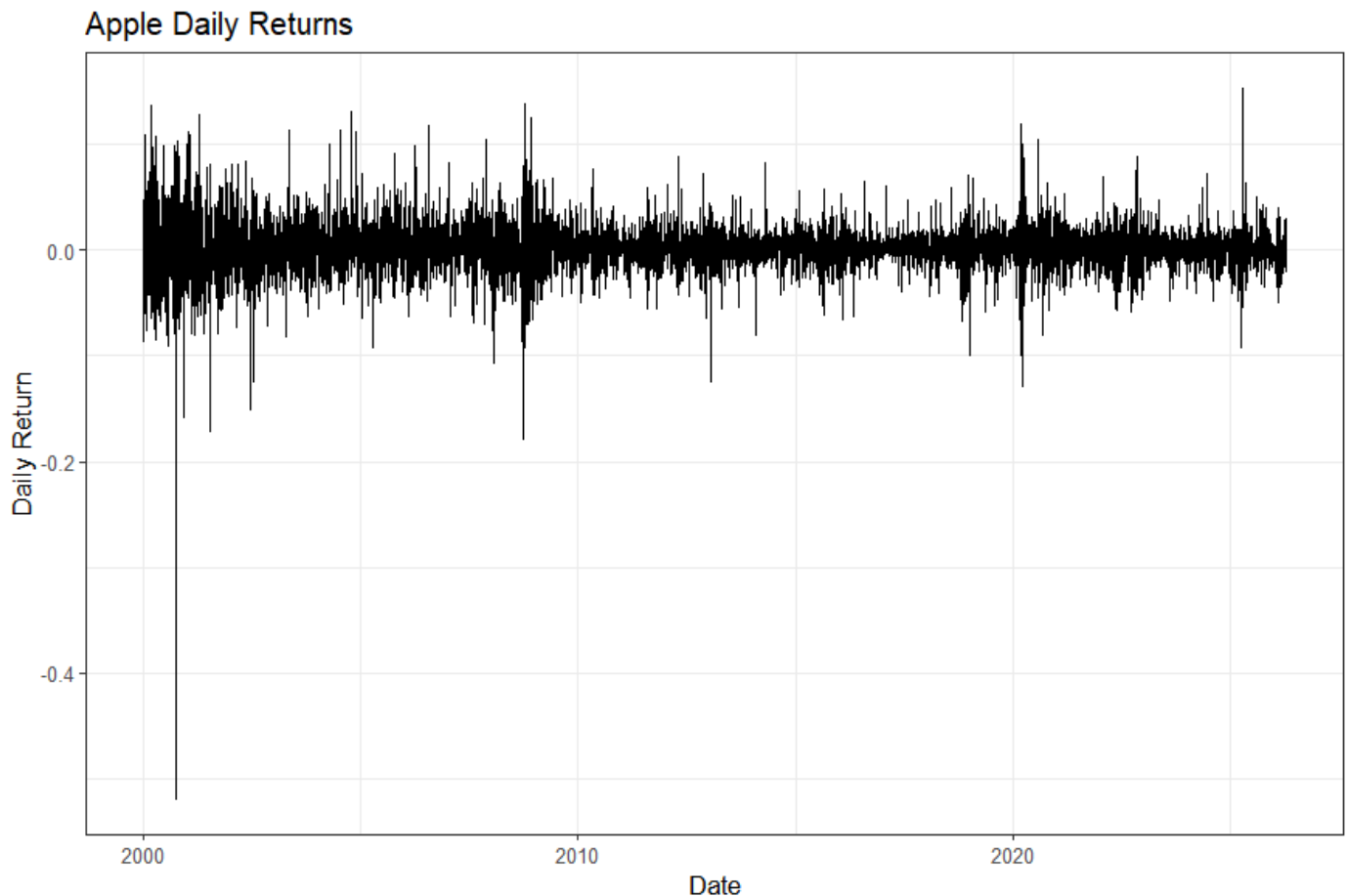
    title = "Apple Daily Returns",

    x = "Date",

    y = "Daily Return"

  ) +

  theme_bw()
```



Now pause and look at the graph.

What do you observe?

You will see:

- Periods where returns fluctuate within a narrow band (calm markets)
- Periods where returns swing dramatically up and down (turbulent markets)
- Importantly: these turbulent days tend to occur **in groups**

This is volatility clustering. High-volatility days are not randomly scattered. They appear in blocks. Low-volatility days also appear in blocks.

This is the defining empirical feature of financial risk.

Step 2: Measuring Realised Volatility

Now that we visually see clusters, we measure them.

A clean way to construct time-varying volatility is:

- Use daily returns
- Group them by month
- Compute the standard deviation within each month

This produces **monthly realised volatility**.

Compute monthly realised volatility

```
library(lubridate)

aapl_monthly_vol <- aapl_daily_ret %>%

  mutate(month = floor_date(date, unit = "month")) %>%

  group_by(month) %>%

  summarise(

    vol_month = sd(daily.returns),

    .groups = "drop"

  )
```

Explanation, line by line:

- `floor_date(date, "month")` converts each day into its month label.
- `group_by(month)` groups all daily returns within the same month.
- `sd(daily.returns)` computes the within-month standard deviation.
- The result is one volatility number per month.

This is called **realised volatility**, because it measures the realised day-to-day fluctuations within that month.

Plot monthly realised volatility

```
ggplot(aapl_monthly_vol, aes(x = month, y = vol_month)) +

  geom_line() +

  labs(

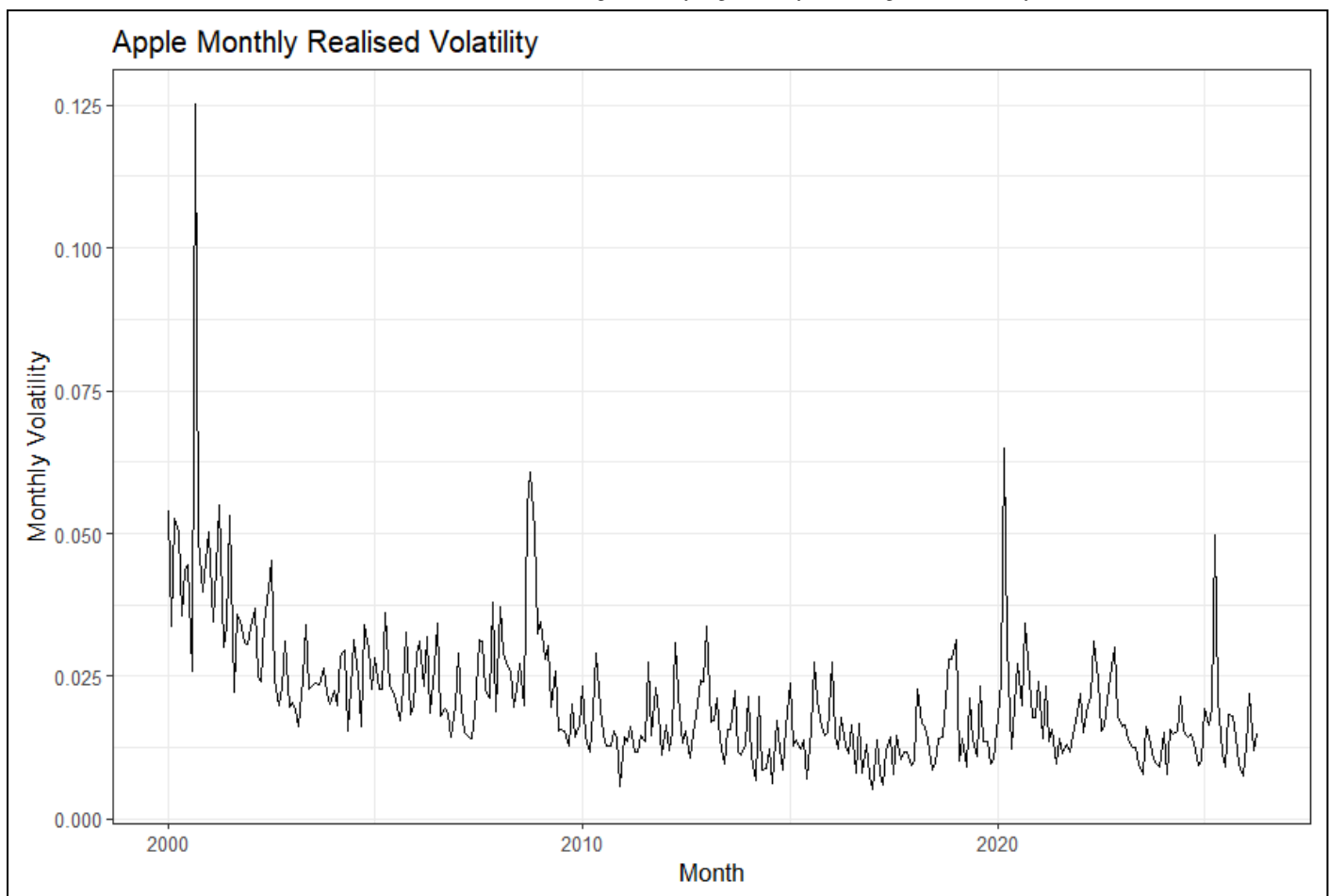
    title = "Apple Monthly Realised Volatility",

    x = "Month",

    y = "Monthly Volatility"

  ) +

  theme_bw()
```



Interpretation:

- Spikes represent high-volatility months.
- Notice that spikes tend to appear in groups.
- Elevated volatility persists for several months during crisis periods.

This confirms clustering visually in a cleaner way than the raw return plot.

Step 3: Squared Returns as a Volatility Proxy

Volatility itself is not directly observed.

A simple and widely used proxy is **squared returns**.

Large return movements generate large squared values, regardless of sign.

Create squared returns

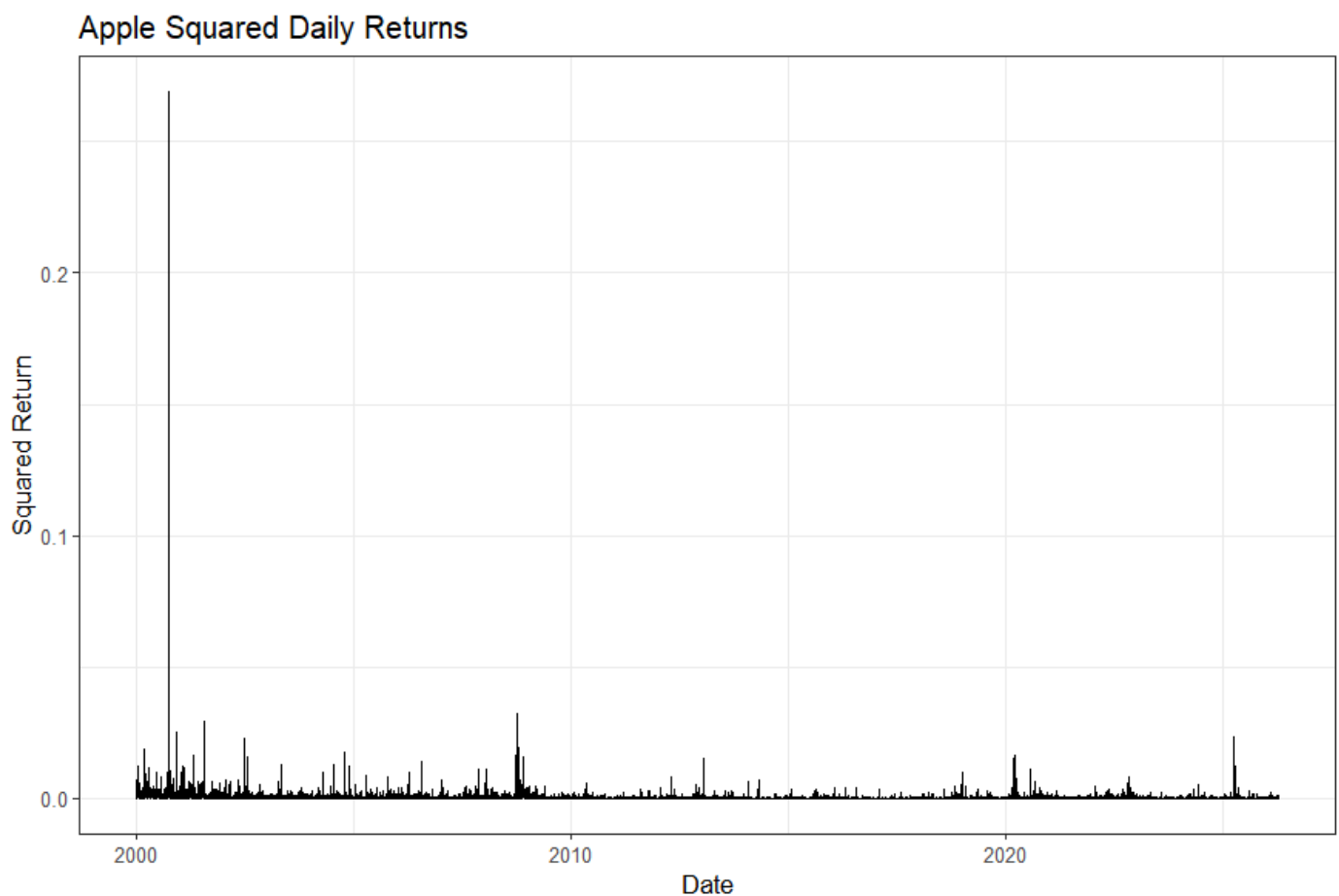
```
aapl_daily_ret <- aapl_daily_ret %>%
  mutate(sq_return = daily_returns^2)
```

Explanation:

- `daily_returns^2` squares each return.
- This removes direction and focuses on magnitude.
- Large positive and negative returns both become large positive numbers.

Plot squared returns

```
ggplot(aapl_daily_ret, aes(x = date, y = sq_return)) +  
  geom_line() +  
  labs(  
    title = "Apple Squared Daily Returns",  
    x = "Date",  
    y = "Squared Return"  
  ) +  
  theme_bw()
```



Interpretation:

- Large spikes represent days with extreme price movements.
- Notice that spikes occur in clusters.
- High values tend to follow high values.

This is direct visual evidence of volatility clustering.

Step 4: Using ACF to Measure Clustering Formally

Visual inspection is powerful. But we can quantify clustering. If volatility clusters, then squared returns today should be related to squared returns yesterday. That relationship can be measured using an **ACF plot**.

Create a vector of squared returns

```
sq <- aapl_daily_ret$sq_return
```

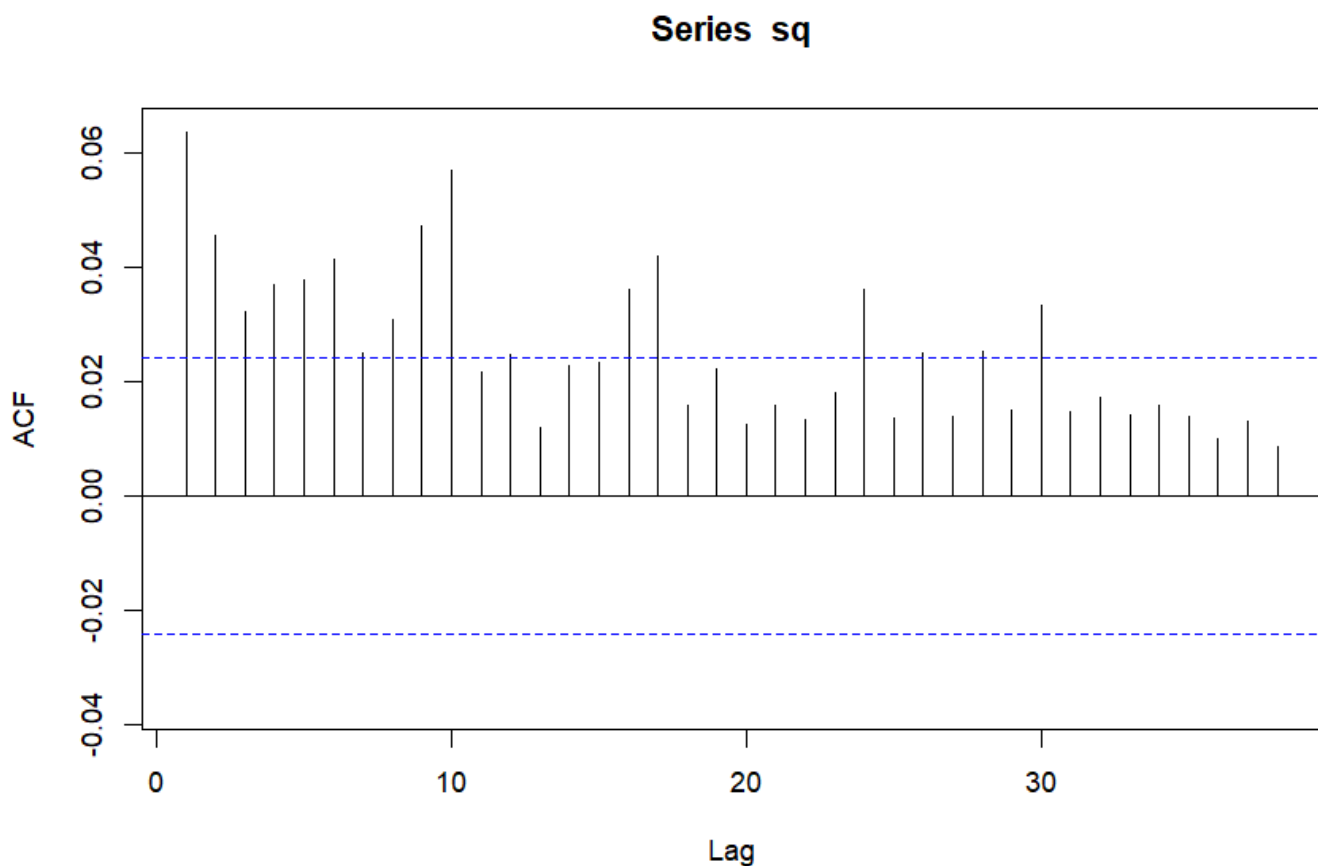
Plot the ACF

```
library(forecast)
```

```
Acf(sq, na.action = na.omit)
```

Explanation:

- `Acf()` computes autocorrelations at different lags.
- Lag 1 compares today's squared return with yesterday's.
- Lag 2 compares today with two days ago.
- If bars remain positive for many lags, volatility is persistent.



Note:

- If volatility were random, correlations would quickly drop to zero.
- In financial data, they typically remain positive across many lags.
- This is statistical evidence of clustering.

What We Have Learned in This Section

We demonstrated volatility clustering using three complementary tools:

1. Raw daily return plots
2. Monthly realised volatility
3. Squared return ACF analysis

The pattern is consistent:

- High-volatility days cluster
- High-volatility months cluster
- Squared returns exhibit persistence

Clustering means that **risk has memory**.